데이터베이스이론 및 실습

# 최종 기말
# 프로젝트 (쿠팡-물류)



첨단IT(빅데이터) 20232656.이은아 / 이시은 교수님

2026.06.17

# Table of Contents


01 요구사항 분석

02 개념적 설계 (ERD)

03 논리적 설계

04 물리적 설계

05 문제

06 알게 된 점, 느낀 점

# 1. 시스템 개요

본 데이터베이스는 쿠팡 물류센터의 알바생, 물류센터, 업무, 근무 정보를 체계적으로 관리하기 위한 시스템이다. 물류센터에서는 여러 알바생이 다양한 업무를 수행할 수 있으며, 알바생은 근무 날짜와 시간에 따라 특정 물류센터와 업무에 배정된다. 따라서 본 시스템은 알바생 정보 관리, 물류센터 정보 관리, 업무 정보 관리, 근무 기록 관리, 급여 관리가 가능하도록 데이터베이스를 구성한다.

본 시스템의 주요 목적은 알바생이 어느 물류센터에서 어떤 업무를 수행했는지 기록하고, 근무 시간과 급여 정보를 관리하는 것이다. 이를 통해 물류센터별 근무 현황, 알바생별 근무 이력, 업무별 급여 지급 내역 등을 확인할 수 있다.

# 2. 주요 관리 대상

본 데이터베이스에서 관리해야 하는 주요 대상은 알바생, 물류센터, 업무, 근무 정보이다.

알바생은 물류센터에서 근무하는 인원으로, 알바생ID, 이름, 전화번호, 근무가능시간, 주소 정보를 가진다.
알바생ID는 각 알바생을 구분하기 위한 고유 식별자이며, 한 명의 알바생은 여러 번 근무할 수 있다.

물류센터는 알바생이 실제로 근무하는 장소를 의미한다. 물류센터는 센터ID, 센터명, 위치 정보를 가진다.
센터ID는 각 물류센터를 구분하기 위한 고유 식별자이며, 하나의 물류센터에서는 여러 알바생의 근무가 발생할 수 있다.

업무는 물류센터에서 수행되는 작업을 의미한다. 업무는 업무ID, 업무명, 업무종류, 시급 정보를 가진다.
업무ID는 각 업무를 구분하기 위한 고유 식별자이며, 하나의 업무는 여러 근무 기록에서 반복적으로 사용될 수 있다.

근무는 알바생이 특정 날짜와 시간에 특정 물류센터에서 특정 업무를 수행한 기록이다.
근무는 근무ID, 근무날짜, 근무시간, 급여, 알바생ID, 센터ID, 업무ID 정보를 가진다. 근무ID는 각 근무 기록을 구분하기 위한 고유 식별자이며,
알바생, 물류센터, 업무와 연결되는 중심 테이블 역할을 한다.

# 3. 엔터티 및 속성 분석

알바생 엔터티는 물류센터에서 근무하는 사람의 기본 정보를 저장한다.

알바생ID는 기본키로 설정하여 알바생을 고유하게 식별한다.
이름은 알바생의 성명을 저장하며, 전화번호는 연락을 위한 정보로 사용된다.

근무가능시간은 알바생이 근무할 수 있는 시간대를 나타내며,
주소는 알바생의 거주 지역 정보를 저장한다.

# 3. 엔터티 및 속성 분석

물류센터 엔터티는 알바생이 근무하는 센터 정보를 저장한다.

센터ID는 물류센터를 구분하는 기본키이며, 센터명은 물류센터의 이름을 의미한다.
위치는 물류센터가 위치한 지역 정보를 저장한다.

# 3. 엔터티 및 속성 분석

업무 엔터티는 물류센터에서 수행되는 업무 정보를 저장한다.

업무ID는 업무를 구분하는 기본키이다. 업무명은 업무의 이름을 나타내며,
업무종류는 세부 업무 유형을 의미한다.

시급은 해당 업무를 수행했을 때 지급되는 시간당 금액을 의미한다.

# 3. 엔터티 및 속성 분석

근무 엔터티는 알바생의 실제 근무 기록을 저장한다.
근무ID는 각 근무 기록을 구분하는 기본키이다.

근무날짜는 근무가 이루어진 날짜를 저장하고, 근무시간은 실제 근무한 시간대를 저장한다.
급여는 해당 근무에 대해 지급된 금액이다.

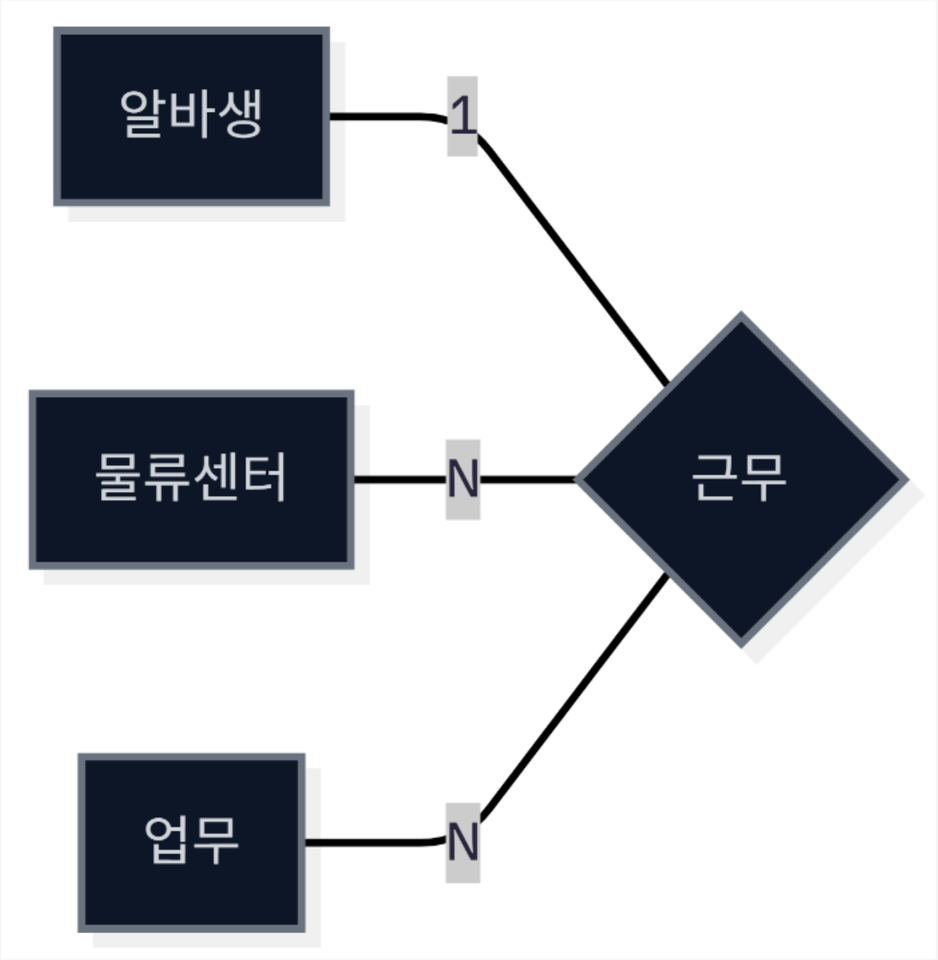
또한 알바생ID, 센터ID, 업무ID를 외래키로 사용하여 알바생, 물류센터, 업무 테이블과 연결된다.

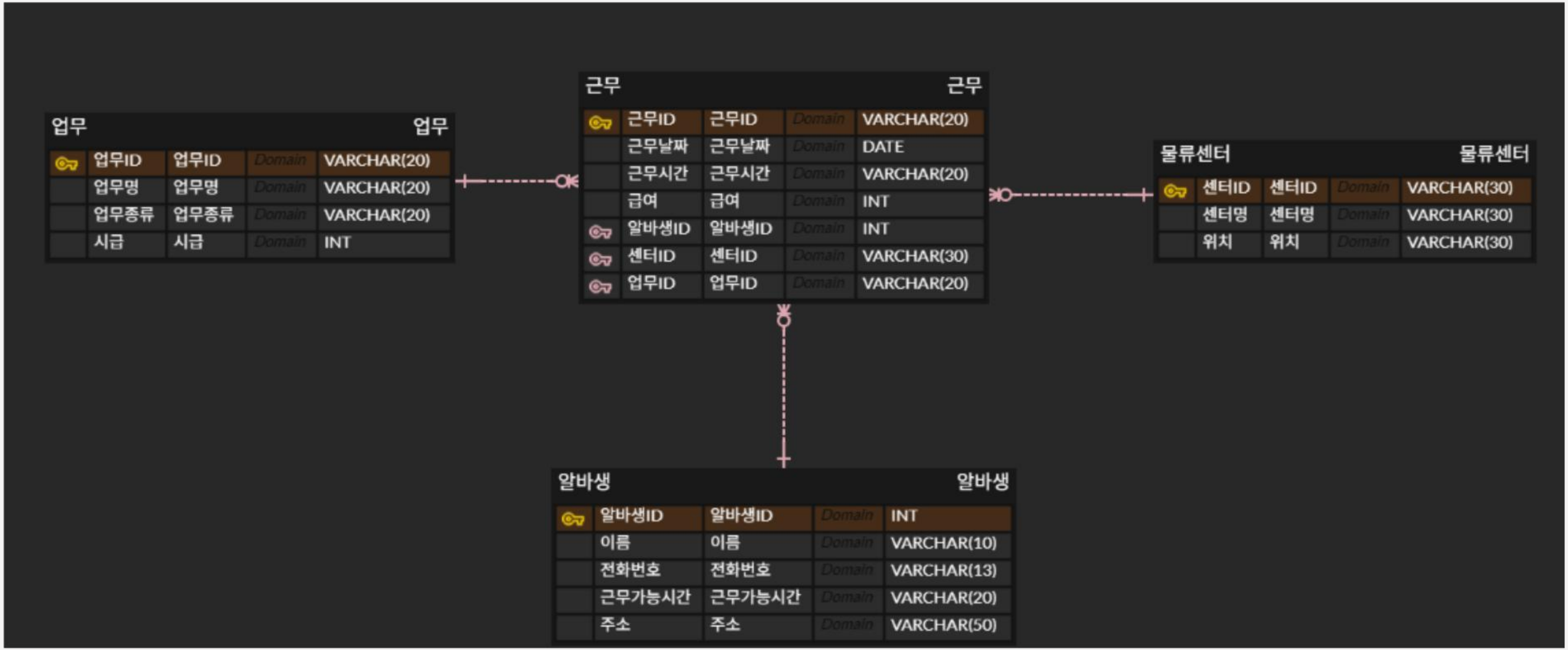
# 3. 엔터티 및 속성 분석

알바생 엔터티는 물류센터에서 근무하는 사람의 기본 정보를 저장한다.

알바생ID는 기본키로 설정하여 알바생을 고유하게 식별한다.
이름은 알바생의 성명을 저장하며, 전화번호는 연락을 위한 정보로 사용된다.

근무가능시간은 알바생이 근무할 수 있는 시간대를 나타내며,
주소는 알바생의 거주 지역 정보를 저장한다.


알바생
1
물류센터
N
근무
업무
N


업무
업무
업무ID 업무ID Domain VARCHAR(20)
업무명 업무명 Domain VARCHAR(20)
업무종류 업무종류 Domain VARCHAR(20)
시급 시급 Domain INT
근무
근무
근무ID 근무ID Domain VARCHAR(20)
근무날짜 근무날짜 Domain DATE
근무시간 근무시간 Domain VARCHAR(20)
급여 급여 Domain INT
알바생ID 알바생ID Domain INT
센터ID 센터ID Domain VARCHAR(30)
업무ID 업무ID Domain VARCHAR(20)
물류센터
물류센터
센터ID 센터ID Domain VARCHAR(30)
센터명 센터명 Domain VARCHAR(30)
위치 위치 Domain VARCHAR(30)
알바생
알바생
알바생ID 알바생ID Domain INT
이름 이름 Domain VARCHAR(10)
전화번호 전화번호 Domain VARCHAR(13)
근무가능시간 근무가능시간 Domain VARCHAR(20)
주소 주소 Domain VARCHAR(50)

| 구분 | 내용 |
| --- | --- |
| 사용 DBMS | MySQL |
| 데이터베이스명 | project |
| 문자셋 | utf8mb4 |
| 주요 테이블 | 알바생, 물류센터, 업무, 근무 |
| 중심 테이블 | 근무 |
| 기본키 | 알바생ID, 센터ID, 업무ID, 근무ID |
| 외래키 | 근무.알바생ID, 근무.센터ID, 근무.업무ID |
| 관계 | 알바생-근무 1:N, 물류센터-근무 1:N, 업무-근무 1:N |

```sql
-- 1. 알바생 테이블 생성
create table 알바생 (
    알바생ID int not null primary key,
    이름 char(5) not null,
    전화번호 int not null,
    근무가능시간 char(20),
    주소 char(30)
);

-- 2. 물류센터 테이블 생성
create table 물류센터 (
    센터ID char(30) not null primary key,
    센터명 char(10),
    위치 char(30)
);
```

```sql
-- 3. 업무 테이블 생성
create table 업무 (
    업무ID char(20) not null primary key,
    업무명 char(10),
    업무종류 char(10),
    시급 int
);

-- 4. 근무 테이블 생성 (외래키 설정 포함)
create table 근무 (
    근무ID char(20) not null primary key,
    근무날짜 char(30),
    근무시간 char(20),
    급여 int,
    알바생ID int not null,
    센터ID char(30) not null,
    업무ID char(20) not null,
    foreign key (알바생ID) references 알바생(알바생ID),
    foreign key (센터ID) references 물류센터(센터ID),
    foreign key (업무ID) references 업무(업무ID)
);
```

```
-- 알바생 데이터 삽입
insert into 알바생 values (20232656, "이은아 ,        1547, "18:00~21:00", "천안 서북구");
insert into 알바생 values (20232657, "이응아", 01[redacted]1548, "19:00~23:00", "천안 동남구");
insert into 알바생 values (20232658, "이은이", 01[redacted]1549, "00:00~07:00", "대전 뭐있냐");

-- 물류센터 데이터 삽입
insert into 물류센터 values ("CK1001", "천안 쿠팡 센터", "천안시");
insert into 물류센터 values ("CK1005", "대전 쿠팡 센터", "대전");
insert into 물류센터 values ("CK1006", "수원 쿠팡 센터", "수원");
insert into 물류센터 values ("CK1007", "인천 쿠팡 센터", "인천");
insert into 물류센터 values ("CK1008", "울산 쿠팡 센터", "울산");
```

```sql
-- 업무 데이터 삽입
insert into 업무 values ("ABG2084", "알바생", "물류", "100000");
insert into 업무 values ("ABG2085", "알바생", "박스", "100000");
insert into 업무 values ("ABG2086", "알바생", "운전", "100000");
insert into 업무 values ("ABG2087", "알바생", "테이핑", "100000");

-- 근무 데이터 삽입 (기존 데이터와 추가 데이터를 골고루 섞어 배치)
-- 컬럼 순서: 근무ID, 근무날짜, 근무시간, 급여, 알바생ID, 센터ID, 업무ID
insert into 근무 values ("G1001", "2026.06.10", "18:00~21:00", 300000, 20232656, "CK1001", "ABG2084");
insert into 근무 values ("G1002", "2026.06.11", "19:00~23:00", 400000, 20232657, "CK1005", "ABG2085");
insert into 근무 values ("G1003", "2026.06.12", "00:00~07:00", 700000, 20232658, "CK1006", "ABG2086");
insert into 근무 values ("G1004", "2026.06.13", "18:00~21:00", 300000, 20232656, "CK1001", "ABG2087");
insert into 근무 values ("G1005", "2026.06.14", "19:00~23:00", 400000, 20232657, "CK1007", "ABG2084");
insert into 근무 values ("G1006", "2026.06.15", "00:00~07:00", 700000, 20232658, "CK1008", "ABG2085");
```

```sql
-- 문제

-- 1. 알바생 테이블의 모든 행과 열을 조회하시오.
SELECT * FROM 알바생;

-- 2. 업무 테이블에 등록된 모든 업무의 이름과 시급을 조회하시오.
SELECT 업무명, 시급 FROM 업무;

-- 3. 물류센터 테이블에서 위치가 '천안시'인 센터의 센터명을 조회하시오.
SELECT 센터명 FROM 물류센터 WHERE 위치 = '천안시';
```

1

| 알바생 ID | 이름 | 전화번호 | 근무가능 시간 | 주소 |
|---|---|---|---|---|
| 20232656 | 이은아 | | 18:00~21:00 | 천안 서북구 |
| 20232657 | 이응아 | | 19:00~23:00 | 천안 동남구 |
| 20232658 | 이은이 | | 00:00~07:00 | 대전 뭐있냐 |
| NULL | NULL | NULL | NULL | NULL |

2

| 업무명 | 시급 |
|---|---|
| 알바생 | 100000 |
| 알바생 | 100000 |
| 알바생 | 100000 |
| 알바생 | 100000 |

3

| 센터명 |
|---|
| 천안 쿠팡 센터 |

```sql
-- 4. 알바생 테이블에서 알바생ID가 20232657인 사원의 이름과 전화번호를 조회하시오.
SELECT 이름, 전화번호 FROM 알바생 WHERE 알바생ID = 20232657;

-- 5. 근무 테이블에서 급여가 400000원인 근무 기록의 모든 정보를 조회하시오.
SELECT * FROM 근무 WHERE 급여 = 400000;

-- 6. 업무 테이블에서 시급이 높은 업무부터 순서대로(내림차순) 정렬하여 조회하시오.
SELECT * FROM 업무 ORDER BY 시급 DESC;
```

4

| | 이름 | 전화번호 |
|---|---|---|
| ▶ | 이응아 | 105€ ... |

5

| | 근무ID | 근무날짜 | 근무시간 | 급여 | 알바생 ID | 센터 ID | 업무 ID |
|---|---|---|---|---|---|---|---|
| ▶ | G1002 | 2026.06.11 | 19:00~23:00 | 400000 | 20232657 | CK1005 | ABG2085 |
| | G1005 | 2026.06.14 | 19:00~23:00 | 400000 | 20232657 | CK1007 | ABG2084 |
| * | NULL | NULL | NULL | NULL | NULL | NULL | NULL |

6

| | 업무ID | 업무명 | 업무종류 | 시급 |
|---|---|---|---|---|
| ▶ | ABG2084 | 알바생 | 물류 | 100000 |
| | ABG2085 | 알바생 | 박스 | 100000 |
| | ABG2086 | 알바생 | 운전 | 100000 |
| | ABG2087 | 알바생 | 테이핑 | 100000 |
| * | NULL | NULL | NULL | NULL |

```sql
-- 7. 알바생 테이블에서 주소에 '천안'이라는 단어가 포함된 알바생의 정보를 조회하시오.
SELECT * FROM 알바생 WHERE 주소 LIKE '%천안%';

-- 8. 근무 테이블에 등록된 전체 근무 기록의 총 개수(근무 건수)를 구하시오.
SELECT COUNT(*) FROM 근무;

-- 9. 근무 테이블에서 지금까지 지급된 알바생 급여의 전체 합계를 구하시오.
SELECT SUM(급여) FROM 근무;

-- 10. 알바생 테이블에서 중복을 제외한 순수 '근무가능시간'의 종류가 몇 개인지 조회하시오.
SELECT COUNT(DISTINCT 근무가능시간) FROM 알바생;
```

7

| 알바생 ID | 이름 | 전화번호 | 근무가능시간 | 주소 |
|---|---|---|---|---|
| 20232656 | 이은아 | 1056... | 18:00~21:00 | 천안 서북구 |
| 20232657 | 이을아 | 1056... | 19:00~23:00 | 천안 동남구 |
| NULL | NULL | NULL | NULL | NULL |

8

| COUNT(*) |
|---|
| 6 |

9

| SUM(급여) |
|---|
| 2800000 |

10

| COUNT(DISTINCT 근무가능시간) |
|---|
| 3 |

```sql
-- 11. 근무 테이블에서 각 알바생ID별로 실행한 총 근무 횟수를 조회하시오.
SELECT 알바생ID, COUNT(*) FROM 근무 GROUP BY 알바생ID;

-- 12. 근무 테이블에서 각 물류센터ID별로 지급된 총 급여액을 구하시오.
SELECT 센터ID, SUM(급여) FROM 근무 GROUP BY 센터ID;

-- 13. 근무 테이블에서 근무 기록을 날짜순으로 정렬하되, 날짜가 같다면 급여가 높은 순으로 정렬하시오.
SELECT * FROM 근무 ORDER BY 근무날짜 ASC, 급여 DESC;

-- 14. 근무 테이블에서 각 업무ID별로 지급된 평균 급여를 구하시오.
SELECT 업무ID, AVG(급여) FROM 근무 GROUP BY 업무ID;
```

11

| 알바생ID | COUNT(*) |
|---|---|
| 20232656 | 2 |
| 20232657 | 2 |
| 20232658 | 2 |

12

| 센터ID | SUM(급여) |
|---|---|
| CK1001 | 600000 |
| CK1005 | 400000 |
| CK1006 | 700000 |
| CK1007 | 400000 |
| CK1008 | 700000 |

13

| 근무ID | 근무날짜 | 근무시간 | 급여 | 알바생ID | 센터ID | 업무ID |
|---|---|---|---|---|---|---|
| G1001 | 2026.06.10 | 18:00~21:00 | 300000 | 20232656 | CK1001 | ABG2084 |
| G1002 | 2026.06.11 | 19:00~23:00 | 400000 | 20232657 | CK1005 | ABG2085 |
| G1003 | 2026.06.12 | 00:00~07:00 | 700000 | 20232658 | CK1006 | ABG2086 |
| G1004 | 2026.06.13 | 18:00~21:00 | 300000 | 20232656 | CK1001 | ABG2087 |
| G1005 | 2026.06.14 | 19:00~23:00 | 400000 | 20232657 | CK1007 | ABG2084 |
| G1006 | 2026.06.15 | 00:00~07:00 | 700000 | 20232658 | CK1008 | ABG2085 |
| NULL | NULL | NULL | NULL | NULL | NULL | NULL |

14

| 업무ID | AVG(급여) |
|---|---|
| ABG2084 | 350000.0000 |
| ABG2085 | 550000.0000 |
| ABG2086 | 700000.0000 |
| ABG2087 | 300000.0000 |

```sql
-- 15. 근무 테이블과 알바생 테이블을 조인하여 근무ID, 근무날짜, 그리고 알바생의 이름을 조회하시오.
SELECT g.근무ID, g.근무날짜, a.이름 FROM 근무 g JOIN 알바생 a ON g.알바생ID = a.알바생ID;

-- 16. 근무 테이블과 물류센터 테이블을 조인하여 '천안 쿠팡 센터'에서 근무한 기록의 근무날짜와 급여를 조회하시오.
SELECT g.근무날짜, g.급여 FROM 근무 g JOIN 물류센터 c ON g.센터ID = c.센터ID WHERE c.센터명 = '천안 쿠팡 센터';

-- 17. '이은아' 알바생이 근무한 날짜와 배정받았던 근무시간을 조회하시오.
SELECT g.근무날짜, g.근무시간 FROM 근무 g JOIN 알바생 a ON g.알바생ID = a.알바생ID WHERE a.이름 = '이은아';

-- 18. '박스' 종류의 업무를 수행한 근무 기록의 근무ID, 급여, 업무종류를 조회하시오.
SELECT g.근무ID, g.급여, j.업무종류 FROM 근무 g JOIN 업무 j ON g.업무ID = j.업무ID WHERE j.업무종류 = '박스';

-- 19. 각 알바생의 이름과 해당 알바생이 지금까지 근무하여 받은 총 누적 급여를 조회하시오.
SELECT a.이름, SUM(g.급여) FROM 근무 g JOIN 알바생 a ON g.알바생ID = a.알바생ID GROUP BY a.이름;
```

15

| 근무ID | 근무날짜 | 이름 |
|---|---|---|
| G1001 | 2026.06.10 | 이은아 |
| G1004 | 2026.06.13 | 이은아 |
| G1002 | 2026.06.11 | 이을아 |
| G1005 | 2026.06.14 | 이을아 |
| G1003 | 2026.06.12 | 이은이 |
| G1006 | 2026.06.15 | 이은이 |

16

| 근무날짜 | 급여 |
|---|---|
| 2026.06.10 | 300000 |
| 2026.06.13 | 300000 |

17

| 근무날짜 | 근무시간 |
|---|---|
| 2026.06.10 | 18:00~21:00 |
| 2026.06.13 | 18:00~21:00 |

18

| 근무ID | 급여 | 업무종류 |
|---|---|---|
| G1002 | 400000 | 박스 |
| G1006 | 700000 | 박스 |

19

| 이름 | SUM(g.급여) |
|---|---|
| 이은아 | 600000 |
| 이을아 | 800000 |
| 이은이 | 1400000 |

```sql
-- 20. 각 물류센터의 센터명과 해당 센터에서 진행된 총 근무 건수를 조회하시오.
SELECT c.센터명, COUNT(g.근무ID) FROM 근무 g JOIN 물류센터 c ON g.센터ID = c.센터ID GROUP BY c.센터명;


-- 21. 알바생 이름, 근무한 물류센터의 센터명, 수행한 업무명을 한 번에 묶어서 조회하시오.
SELECT a.이름, c.센터명, j.업무명 FROM 근무 g JOIN 알바생 a ON g.알바생ID = a.알바생ID JOIN 물류센터 c ON g.센터ID = c.센터ID JOIN 업무 j ON g.업무ID = j.업무ID;

-- 22. '이은아' 알바생이 근무한 물류센터의 센터명과 해당 센터의 실제 위치를 조회하시오.
SELECT c.센터명, c.위치 FROM 근무 g JOIN 알바생 a ON g.알바생ID = a.알바생ID JOIN 물류센터 c ON g.센터ID = c.센터ID WHERE a.이름 = '이은아';
```

20

| 센터명 | COUNT(g.근무ID) |
|---|---|
| 천안 쿠팡 센터 | 2 |
| 대전 쿠팡 센터 | 1 |
| 수원 쿠팡 센터 | 1 |
| 인천 쿠팡 센터 | 1 |
| 울산 쿠팡 센터 | 1 |

| 이름 | 센터명 | 업무명 |
|---|---|---|
| 이은아 | 천안 쿠팡 센터 | 알바생 |
| 이은아 | 천안 쿠팡 센터 | 알바생 |
| 이응아 | 대전 쿠팡 센터 | 알바생 |
| 이응아 | 인천 쿠팡 센터 | 알바생 |
| 이은이 | 수원 쿠팡 센터 | 알바생 |
| 이은이 | 울산 쿠팡 센터 | 알바생 |

21

| 센터명 | 위치 |
|---|---|
| 대전 쿠팡 센터 | 대전 |
| 인천 쿠팡 센터 | 인천 |

22

```sql
-- 23. 근무 테이블에서 총 누적 급여 합계가 700,000원 이상인 알바생의 ID와 총 급여를 조회하시오.
SELECT 알바생ID, SUM(급여) FROM 근무 GROUP BY 알바생ID HAVING SUM(급여) >= 700000;

-- 24. 업무 테이블에서 가장 시급이 높은 업무를 수행한 근무 기록의 근무ID와 급여를 서브쿼리를 사용하여 조회하시오.
SELECT 근무ID, 급여 FROM 근무 WHERE 업무ID = (SELECT 업무ID FROM 업무 ORDER BY 시급 DESC LIMIT 1);

-- 25. '운전' 업무를 수행한 알바생들의 이름과 전화번호를 중복 없이 조회하시오.
SELECT DISTINCT a.이름, a.전화번호 FROM 근무 g JOIN 알바생 a ON g.알바생ID = a.알바생ID JOIN 업무 j ON g.업무ID = j.업무ID WHERE j.업무종류 = '운전';
```

23

| 알바생ID | SUM(급여) |
|---|---|
| 20232657 | 800000 |
| 20232658 | 1400000 |

| 근무ID | 급여 |
|---|---|
| G1001 | 300000 |
| G1005 | 400000 |
| NULL | NULL |

24

| 이름 | 전화번호 |
|---|---|
| 이은이 | 1056101549 |

25

이번 데이터베이스 이론 및 실습 수업을 통해 저는 데이터베이스에 대한 기본 개념과 SQL 활용 방법을 보다 체계적으로 이해할 수 있었습니다. 수업을 듣기 전에는 데이터베이스라는 개념은 알고 있었지만, 테이블을 구성하는 방식이나 기본키와 외래키의 역할, SQL에서 사용되는 명령어와 함수의 정확한 의미에 대해서는 깊이 있게 알지 못했습니다. 또한 문제를 해결할 때 어떤 순서로 접근해야 하는지에 대한 기준이 부족하여 SQL 문제를 풀 때마다 어렵게 느껴지는 부분이 많았습니다.

특히 이번 수업을 통해 가장 확실하게 이해하게 된 부분은 외래키의 개념과 활용 방법입니다. 테이블을 만들 때 외래키를 어떻게 설정해야 하는지, 왜 필요한지, 테이블 간의 관계를 어떻게 연결해야 하는지에 대해 이전에는 많이 헷갈렸습니다. 하지만 수업을 통해 기본키와 외래키의 관계를 반복적으로 배우고 직접 문제에 적용하면서, 외래키가 데이터의 무결성을 유지하고 테이블 간의 관계를 표현하는 중요한 역할을 한다는 점을 알게 되었습니다. 이를 통해 데이터베이스 설계에서 관계를 정확하게 설정하는 것이 얼마나 중요한지도 이해할 수 있었습니다.

또한 중첩 질의문과 같은 부분도 처음에는 어렵게 느껴졌습니다. 이전에는 자신이 없는 유형의 문제를 피하거나, 간단한 문제 위주로만 풀려고 했던 경우가 많았습니다. 그러나 교수님께서 개념을 반복해서 설명해주시고 다양한 문제를 풀 수 있도록 지도해주신 덕분에, 어려웠던 부분도 조금씩 이해할 수 있었습니다. 반복 학습을 통해 처음에는 막막했던 SQL문도 직접 작성할 수 있게 되었고, 문제를 해결하는 데 대한 자신감도 생겼습니다.

이번 수업을 통해 느낀 점은 코딩이나 데이터베이스 공부는 단순히 개념을 듣는 것만으로 끝나는 것이 아니라, 직접 작성하고 반복해서 문제를 풀어보는 과정이 중요하다는 것입니다. 알고 있다고 생각했던 내용도 실제 문제에 적용해보면 부족한 부분이 드러났고, 그 과정을 반복하면서 조금씩 제 실력으로 자리 잡게 되었습니다. 특히 이번 수업에서 배운 SQL 작성 방법과 데이터베이스 설계 과정은 앞으로 정보처리기사 준비나 다른 개발 프로젝트를 진행할 때에도 큰 도움이 될 것이라고 생각합니다.

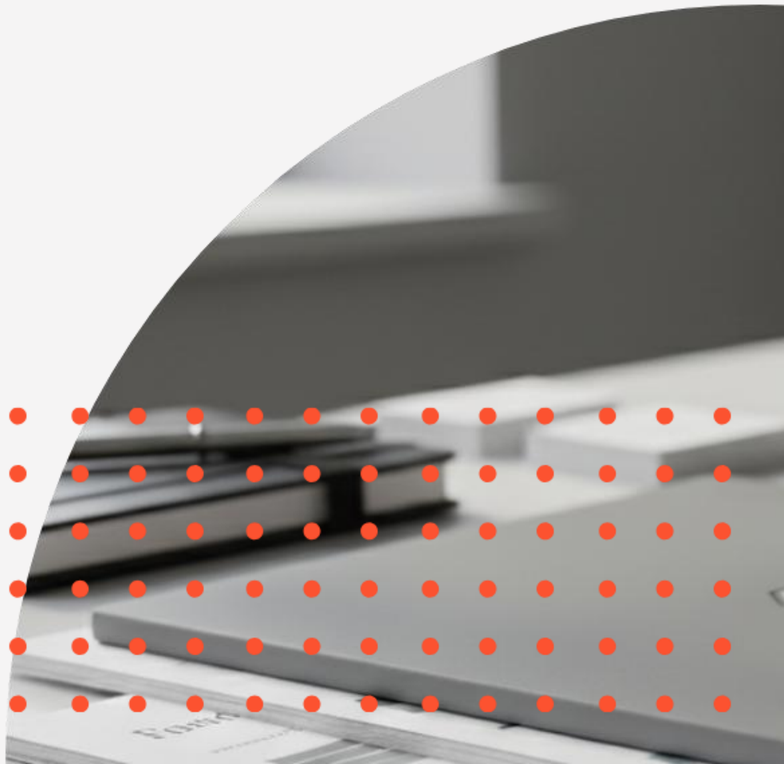
마지막 프로젝트를 진행하면서 직접 요구사항을 정리하고, 개념적 설계와 논리적 설계, 물리적 설계를 거쳐 데이터베이스를 완성해보는 경험도 의미 있었습니다. 단순히 주어진 문제를 푸는 것이 아니라, 제가 직접 주제를 정하고 필요한 테이블과 관계를 설계하면서 데이터베이스가 실제로 어떤 과정으로 만들어지는지 이해할 수 있었습니다. 그동안 수업에서 배운 내용을 최종 프로젝트에 적용해볼 수 있었고, 저만의 데이터베이스를 만들어보는 과정이 흥미롭고 보람 있었습니다.

저한테 있어서 이 데이터베이스 이론 및 실습 수업은 저에게 데이터베이스에 대한 이해도를 높여준 의미 있는 수업이었습니다.
교수님의 반복적인 설명과 문제 풀이 중심의 지도 방식 덕분에 어려웠던 개념도 포기하지 않고 이해할 수 있었으며,
SQL을 직접 작성하는 능력도 향상될 수 있었습니다. 감사합니다.

앞으로도 이번 수업에서 배운 내용을 바탕으로 데이터베이스를 더 깊이 공부하고, 다양한 프로젝트에 활용할 수 있도록 노력하고자 합니다. 또한 교수님께서 수업 시간에 알려주신 내용을 토대로 정보처리기사 자격증 공부까지 이어가며, 최종적으로 자격증 취득을 목표로 꾸준히 학습하고자 합니다.

# 감사합니다.





## contact

데이터베이스 이론 및 실습
담당 교수: 이시은 교수님
학번: 20232656
이름: 이은아